

AuthGuard ML — AI-Based User Authentication Anomaly Detection System

Professional Project Documentation

Project Title: AI-Based User Authentication Anomaly Detection System
System Name: AuthGuard ML
Dataset Reference: CMU Insider Threat Test Dataset (https://kilthub.cmu.edu/articles/dataset/Insider_Threat_Test_Dataset/12841247/1)
Technology Stack: React, TypeScript, Vite, Supabase (PostgreSQL), Tailwind CSS
Date: September 2026

Table of Contents

- [Chapter One — Introduction](#chapter-one-introduction)
- [Chapter Two — Literature Review](#chapter-two-literature-review)

wo
--lit
era
tur
e-r
evi
ew
)

- [Chapter Three — Project Management & Methodology](#chapter-three--project-management--methodology)
- [Chapter Four — System Analysis,

Design & Security] (#chapter-for-system-analysis-design--security)

- [Chapter Five — AI/ML and Software Implementation](#chapter-five-ai-ml-and-software-implementation)

tion)

- [Chapter Six

— Testing, Quality & Security] (#chapter-six-testing-quality-security)

- [Chapter Seven

— Deployment, Monitoring & Project Control] (#chapter-seven-deployment-monitoring-project-control)

en-
-de
plo
ym
ent
-m
oni
tori
ng-
-pr
oje
ct-
co
ntr
ol)

- [Ch
apt
er
Eig
ht
—
Co
ncl
usi
on]
(#c
ha
pte
r-ei
ght
--c
on
clu
sio
n)

Chapter One — Introduction

1.1 Background

User authentication systems — login via username/password, tokens, or biometrics — are the primary gateway to digital accounts and services. However, credentials are frequently compromised through phishing, credential stuffing, data breaches, or malware, allowing attackers to gain unauthorized access using legitimate login credentials. Once inside, an attacker's behavior (login time, location, device fingerprint, IP address, typing patterns, navigation habits) often differs subtly or significantly from that of the genuine account owner.

Traditional authentication systems verify identity at the point of login but lack the intelligence to continuously assess whether the behavior following login is consistent with the legitimate user, leaving a critical blind spot in account security. This gap between initial authentication and continuous behavioral verification is where the majority of account compromise incidents go undetected.

1.2 Problem Statement

Attackers may compromise legitimate accounts and behave differently from genuine users. There is a need for an ML system that identifies unusual login behavior by continuously analyzing post-authentication activity patterns and flagging deviations from established behavioral baselines.

The core problem is that conventional authentication is a binary, point-in-time check — either the credential matches or it does not. It does not account for the scenario where an attacker possesses

valid credentials but exhibits behavioral patterns inconsistent with the legitimate account owner. This creates a persistent detection gap that attackers exploit for extended periods, often causing significant damage before being discovered.

1.3 Aim

To develop an AI-based user authentication anomaly detection system that continuously monitors login behavior, establishes per-user behavioral baselines, and identifies anomalous activities indicative of account compromise using machine learning techniques.

1.4 Objectives

1.

Data

Collect
on:
Generate
and
collect
login
event
data
model
on
the
CM
U In
side
r Th
reat
Test
t Da
tase
t, ca
pturi
ng u
ser I
Ds,
time
sta
mps
, de
vice
ide
ntifi
ers,
IP a
ddr
ess
es,
and
acti
vity
type
s.

2.

Data

Pre
proc
essi
ng:
Cle
an
and
pre
proc
ess
raw
logi
n ev
ents
, ha
ndli
ng t
emp
oral
feat
ures
, cat
ego
rical
vari
able
s,
and
deri
ving
beh
avio
ral i
ndic
ator
s
(off-
hou
rs fl
ags,
we
eke
nd fl
ags,

day-
of-w
EEK
enc
odin
g).

3.

E
xplo
rato
ry D
ata
Ana

lysis
(ED
A):
Ana
lyze
logi
n pa
tter
ns t
hro
ugh
visu
aliz
atio
ns i
nclu
ding
hou
r/
day
distr
ibuti
ons,
acti
vity
heat
map
s, s
catt
er pl
ots,
and
tren
d an
alysi
s to
iden
tify
patt
erns

and
outli
ers.

4. F
eatu
re E
ngin
eeri
ng:
Tran
sfor
m
raw
logi
n ev
ents

into
num

erical features capturing behavioral deviations: household deviation, PC familiarity, IP familiarity, off-hours indicators, weekend indicators, and day deviation.

5.

Model Selection and Training: Implementation and training multiple unsupervised

ed a
nom
aly
dete
ctio
n al
gorit
hms
— I
sola
tion
For
est,
One
-Cla
ss S
VM,
Loc
al O
utlie
r Fa
ctor,
Aut
oen
cod
er,
and
Rob
ust
Z-S
core
—
and
com
par
e th
eir p
erfo
rma
nce.
6.

Model
Evaluation
: Ev
alua
te tr
aine
d m
odel
s us
ing
acc
urac
y, pr
ecisi
on,
reca
ll,
F1-s
core

, R
OC-
AU
C,
and
conf
usio
n m
atric
es
to s
elec
t
the
best
-per
form
ing
algo
rith
m.

7. D
eplo
yme
nt:
Dep
loy
the
dete
ctio
n sy
ste
m
as
an i
nter
acti
ve
web
app
licati
on
with

real-
time
ano
mal
y sc
orin
g
and
alert
gen
erati
on c
apa
biliti
es.

8. Monitoring and Maintenance: Implement a monitoring dashboard with a alert queue management, severity classification, and acknowledgment workflow for ongoing operational use.

1.5 Significance

- Security Enhancement: Closes

the
po
st-
aut
he
nti
cat
ion
de
tec
tio
n g
ap
by
co
nti
nu
ou
sly
ev
alu
ati
ng
us
er
be
ha
vio
r a
gai
nst
es
tab
lish
ed
ba
seli
ne
s,
pro
vidi
ng
def
en
se-
in-
de
pth
be
yo
nd
poi
nt-
in-t
im
e c
red
ent
ial
ch

ec
ks.

-

Ea
rly
Thr
eat
D
ete
cti
on:
Id
ent
ifie
s c
om
pro
mi
se
d a
cc
ou
nts
thr
ou
gh
be
ha
vio
ral
de
via
tio
ns
bef
ore
si
gni
fic
ant
da
ma
ge
oc
cur
s, r
ed
uci
ng
dw
ell
tim
e a
nd
mi
ni
mi
zin
g i
mp

act

.

•

Ad
apt
ive
B
as
eli
ne
s:
Pe
r-u
ser
be
ha
vio
ral
pro
filin
g a
da
pts
to
ind
ivid
ual
w
ork
pa
tter
ns,
re
du
cin
g f
als
e p
osi
tiv
es
co
mp
are
d
to
on
e-s
ize
-fit
s-
all
rul
e-b
as
ed
sys
te
ms

.

- Operational Integration: Provide security analysis with a structured alert queue, severity classification, and investigation workflow for efficient incident response.

- Educational Value: Demonstrate the complete ML project lifecycle — from data collection through deployment and monitoring — in a single integrated system.

1.6 Scope

In-Scope:

- Sy
nth
eti
c l
ogi
n e
ve
nt
ge
ner
ati
on
mo
del
ed
on
C
M
U I
nsi
der
Th
rea
t d
ata
set
str
uct
ure
- Be
ha
vio
ral
fea
tur
e e
ngi
ne
eri
ng
for
9 d
eri
ve
d f
eat
ure
s p
er l
ogi
n e
ve
nt
-

Training and evaluation of 5 unsupervised anomaly detection algorithms

- Web-based dashboard with 8 functional pages covering the full IMLP pipeline
- Real-time anomaly detection

ect
ion
wit
h li
ve
ev
ent
str
ea
mi
ng

- Alert
generation,
se
ver
ity
cla
ssif
ica
tio
n,
an
d a
ck
no
wle
dg
me
nt
wo
rkfl
ow

- Su
pa
ba
se
se
(P
ost
gre
SQ
L)
dat
ab
as
e
for
dat
a p
ers
ist
en
ce

Out-of-Scope:

- Bio
me
tric

authentication
(fingerprint, facial
recognition)

- Keystroke dynamics
typing pattern analysis

- Network packet-level intrusion detection

- Integration with external authentication

ent
ica
tio
n p
rov
ide
rs (
OA
uth
, S
AM
L)

- Mo
bil
e a
ppl
ica
tio
n d
epl
oy
me
nt

- Mu
liti-t
en
ant
or
ent
erp
ris
e-s
cal
e d
epl
oy
me
nt

1.7 Limitations

1. S
ynth
etic
Dat
a: T
he s
yste
m u
ses
proc
edu
rally
gen
erat
ed s
ynth
etic

data
model
ded
on
the
CM
Ud
atas
et st
ruct
ure
rath
er t
han
real-
world
d lo
gin l
ogs,
whi
ch
may
not
capt
ure
all
real-
world
d be
havi
oral
com
plex
ities

.

2.

Model
Simplification
: Th
e an
oma
ly d
etec
tion
algo
rith
ms
use
rule
-bas
ed t
hres
hold
app
roxi
mati
ons
of th
eir

ML
cou
nter
part
s to
ena
ble
in-br
ows
er e
xec
utio
n wi
thou
t a
Pyt
hon
bac
ken
d, w
hich
ma
y pr
odu
ce s
light
ly di
ffere
nt r
esul
ts th
an
full li
brar
y im
ple
men
tatio
ns.

3. Si
ngle
-Ten
ant
Arc
hite
ctur
e: T
he c
urre
nt d
eplo
yme
nt
is si
ngle
-ten
ant
with
out
user

authentication, suitable for demonstration and single-team use but not for multi-organization deployment.

4. Feature Scope: Behavioral features are limited to temporal and device-based indicators; richer features such as session duration,

navigati
on p
atte
rns,
and
data
acc
ess
patt
erns

are
not i
nclu
ded.

5. Scalability : The in-browser computation approach is suitable for datasets of ~ 5,000 events ; production-scale deployment with millions of events would require server-side model

infe
renc
e.

1.8 Definitions

Chapter Two — Literature Review

2.1 AI/ML Concepts

2.1.1 Anomaly Detection

Anomaly detection is the identification of data points, events, or observations that deviate significantly from a dataset's normal behavior. Anomaly detection techniques fall into three primary categories:

- Supervised: Requires labeled training data with both normal and malicious examples. Classification algorithms (Random

For
est
, S
VM
, N
eur
al
Ne
tw
ork
s)
are
tra
ine
d
to
dis
tin
gui
sh
bet
we
en
cla
ss
es.
Li
mit
ati
on:
an
om
alo
us
dat
a
is t
ypi
call
y r
are
an
d d
iffic
ult
to l
ab
el.

•

Un
su
per
vis
ed:
O
per
ate
s
on
unl

able
elementary data
, assuming
anomalies
are a
minority.
Algorithms
include
Isolation
Forest
, One-
Class
SVM,
Local
Outlier
Factor,
and
Autoencoders.
These
are the
most
practical

for
insi
der
thr
eat
de
tec
tio
n b
ec
au
se
lab
ele
d a
tta
ck
dat
a
is s
car
ce.

•

Se
mi-
Su
per
vis
ed:
Tr
ain
s
on
nor
ma
l d
ata
on
ly, t
he
n fl
ag
s d
evi
ati
on
s fr
om
th
e l
ear
ne
d n
or
ma
l m
od
el.
Aut
oe

nc
od
ers
tra
ine
d
on
nor
ma
l b
eh
avi
or f
all i
nto
thi
s c
ate
gor
y.

2.1.2 Algorithms Used in This Project

Isolation Forest: An ensemble method that isolates anomalies by randomly selecting a feature and splitting on a random value. Anomalies, being few and different, require fewer splits to isolate, resulting in shorter path lengths in the isolation trees. The anomaly score is derived from the average path length across all trees (Liu, Ting, & Zhou, 2008).

One-Class SVM: A Support Vector Machine variant that learns a decision boundary enclosing the normal data in a high-dimensional feature space. Points falling outside the boundary are classified as anomalies. It uses a kernel function to map data to a higher-dimensional space where separation is possible (Schölkopf et al., 2001).

Local Outlier Factor (LOF): A density-based method that measures the local deviation of density of a data point with respect to its neighbors. Points with substantially lower density than their neighbors are considered outliers. LOF is effective for datasets with varying densities (Breunig, Kriegel, Ng, & Sander, 2000).

Autoencoder: A neural network architecture that compresses input data into a lower-dimensional representation (encoder) and reconstructs it (decoder). When trained on normal data, the model learns to reconstruct normal patterns well but produces high reconstruction error for anomalous inputs. The reconstruction error serves as the anomaly score (Sakurada & Yairi, 2014).

Robust Z-Score: A statistical method using the Median and Median Absolute Deviation (MAD) instead of mean and standard deviation, making it robust to outliers in the training data. Points with a robust Z-score exceeding a threshold are flagged as anomalies (Iglewicz & Hoaglin, 1993).

2.1.3 Evaluation Metrics

-

Ac
cur
ac
y: (
TP
+
TN
) / (
TP
+
TN
+
FP
+
FN
)

— Overall correctness, but misleading with himbalanced datasets.

- Precision: $\frac{TP}{TP + FP}$ — Of all predicted anomalies, how many were real anomalies.
- Recall (Sensitivity)

siti
vity
) : T
P /
(T
P
+ F
N)
—
Of
all
rea
l a
no
ma
lies
, h
ow
m
an
y w
ere
de
tec
ted
.

•

F1-
Sc
ore
: 2
× (
Pr
eci
sio
n
×
Re
call
) / (
Pr
eci
sio
n
+
Re
call
)
—
Ha
rm
oni
c
me
an
of
pre
cisi
on
an

dr
ec
all.

-

ROC-AUC: Area under the Receiver Operating Characteristic curve — Measures the model's ability to distinguish between classes across all thresholds.

For anomaly detection in security contexts, recall is often prioritized over precision — it is better to investigate a false alarm than to miss a real attack.

2.2 Software Engineering / Project Management Concepts

2.2.1 Agile/Scrum Methodology

The project follows an Agile development approach using Scrum principles. Agile was chosen over Waterfall because:

- The ML pipeline requires iterative development with different algorithms and features
- Visualization and dashboard requirements evolved throughout

development

- Continuous integration of new hardware components and packages benefited from short iteration cycles
- The 8-stage pipeline is naturally decomposed into sprint-si

ze
d d
eliv
era
ble
s

Scrum elements applied:

- Product Backlog: 8 pipelines, 5 tags, 5 episodes, 5 Data Collection, 5 EDAs, 5 Feature Engineering, 5 Models, 5 Training, 5 Model Evaluation, 5 Anomalies

ly
De
tec
tio
n,
Mo
nit
ori
ng)

- Sprint
Goal
: Deliver
one
effun
ction
on
all
pipeline
stages
end-to-end
- Increment
: Each
page
is
a shippa
ble
in
crement
with
working
UI
and
database

ke
nd
int
egr
ati
on

-

De
fini
tio
n
of
Do
ne:
Fe
atu
re i
mp
le
me
nte
d, t
yp
e-c
he
ck
ed,
bu
ild
pa
ss
es,
int
egr
ate
d
wit
h s
tat
e
ma
na
ge
me
nt

2.2.2 Work Breakdown Structure (WBS)

The project decomposes into the following WBS hierarchy:

```

AuthGuard ML
% % % 1. Project Initiation
%   % % % 1.1 Problem definition
%   % % % 1.2 Requirements gathering
%   % % % 1.3 Scope definition
% % % 2. Database & Backend
%   % % % 2.1 Schema design (4 tables)
%   % % % 2.2 Migration creation
%   % % % 2.3 RLS policy configuration
%   % % % 2.4 Supabase client setup
% % % 3. Core Libraries
%   % % % 3.1 Type definitions
%   % % % 3.2 Data generator (CMU-style synthetic data)
%   % % % 3.3 Feature engineering engine
%   % % % 3.4 Anomaly detection engine
%   % % % 3.5 Model training & evaluation
%   % % % 3.6 State management store
% % % 4. Chart Components
%   % % % 4.1 Bar chart
%   % % % 4.2 Line chart
%   % % % 4.3 Donut chart
%   % % % 4.4 Heatmap
%   % % % 4.5 Scatter plot
%   % % % 4.6 Gauge chart
%   % % % 4.7 Radar chart
%   % % % 4.8 Confusion matrix
%   % % % 4.9 Metric card
% % % 5. UI Pages
%   % % % 5.1 Dashboard overview
%   % % % 5.2 Data Collection page
%   % % % 5.3 EDA page
%   % % % 5.4 Feature Engineering page
%   % % % 5.5 Model Training page
%   % % % 5.6 Model Evaluation page
%   % % % 5.7 Anomaly Detection page
%   % % % 5.8 Monitoring page
% % % 6. Layout & Navigation
%   % % % 6.1 Sidebar navigation
%   % % % 6.2 Header with alert badge
%   % % % 6.3 Responsive layout
% % % 7. Integration & Testing
%   % % % 7.1 State management wiring
%   % % % 7.2 Database integration
%   % % % 7.3 Build verification
%   % % % 7.4 Type checking
% % % 8. Documentation
%   % % % 8.1 README documentation
%   % % % 8.2 Code comments

```

2.3 Cybersecurity Concepts

2.3.1 Insider Threat Detection

Insider threat detection focuses on identifying malicious activity originating from within an organization. The CERT Division of Carnegie Mellon University's Software Engineering Institute developed the Insider Threat Center and associated datasets to study this problem domain. The CMU Insider Threat Test Dataset provides synthetic but realistic log data capturing user activities including logon/logoff events, web browsing, file access, and email usage.

Key insider threat indicators include:

- Te

mp
ora
l a
no
ma
lies
: A
cc
es
s
at
un
us
ual
ho
urs
or
da
ys

- Sp
ati
al
an
om
ali
es:
Ac
ce
ss
fro
m
unf
am
ilia
r d
evi
ce
s
or
IP
ad
dre
ss
es

- Vol
um
e a
no
ma
lies
: U
nu
su
al f
req
ue
nc
y

or
vol
um
e
of
act
iviti
es

-

Be
ha
vio
ral
an
om
ali
es:
D
evi
ati
on
fro
m
est
abl
ish
ed
us
ag
e p
att
ern
s

2.3.2 Authentication Security

Traditional authentication mechanisms:

-

Kn
owl
ed
ge-
ba
se
d:
Pa
ss
wo
rds
, P
INs
, s
ec
urit
y q
ue
sti
on
s
—
vul

ner
abl
e
to
phi
shi
ng,
br
ute
for
ce,
cr
ed
ent
ial
stu
ffin
g

- Po
ss
es
sio
n-b
as
ed:
To
ke
ns,
s
ma
rt c
ard
s,
mo
bil
e a
uth
ent
ica
tor
s
—
vul
ner
abl
e
to t
hef
t, c
lon
ing
- Bio
me
tric
: Fi
ng
erp
rint

s, f
aci
al r
ec
og
niti
on
—
diff
icul
t
to r
ev
ok
e,
pot
ent
ial
for
fal
se
po
siti
ve
s

Continuous Authentication: An emerging paradigm that verifies identity throughout a session by monitoring behavioral patterns, rather than only at login time. AuthGuard ML implements this concept by evaluating each login event against the user's behavioral baseline.

2.3.3 Secure SDLC

The Secure Software Development Life Cycle integrates security practices into every phase of development:

- Re
qui
re
me
nts
: S
ec
urit
y r
eq
uir
em
ent
s d
efi
ne
d a
lon
gsi
de
fun
cti
on
al r
eq
uir
em

ent
s

- Design: Threat modeling and security architecture
- Implementation: Secure coding practices, input validation
- Testing: Security testing, vulnerability assessments

sm
ent

- De
plo
ym
ent
: S
ec
ure
co
nfi
gur
ati
on,
m
oni
tori
ng
- Ma
int
en
an
ce:
In
cid
ent
re
sp
on
se,
pa
tch
m
an
ag
em
ent

2.4 Related Systems

2.5 Gap Analysis

Chapter Three — Project Management & Methodology

3.1 Project Charter

Project Justification: Existing authentication systems verify identity only at login time, creating a blind spot for attackers using stolen credentials. An ML-based continuous behavioral monitoring system addresses this gap by detecting post-authentication anomalies indicative of account compromise.

3.2 Stakeholder Identification

3.3 Requirements Engineering

3.3.1 Functional Requirements

3.3.2 Non-Functional Requirements

3.3.3 Security Requirements

3.4 Scope Definition

Project Scope: Build a complete, interactive web application that demonstrates the full ML project lifecycle for authentication anomaly detection — from synthetic data generation through real-time detection and monitoring — with persistent storage and professional visualizations.

Product Scope: An 8-page single-page application with sidebar navigation, custom SVG charts, a behavioral profiling engine, 5 anomaly detection algorithms, model evaluation with standard ML metrics, real-time detection mode, and an alert management system.

3.5 Resource Planning and Responsibility Allocation

3.6 Effort/Cost Estimation

Cost: \$0 — All tools and services are free-tier or open-source.

3.7 Scheduling, Milestones and Gantt Chart

Gantt Chart (Text Representation):

```
Week 1: [DB & Libs] [Charts  ] [Layout  ] [Data/EDA]
Week 2: [ML Pages  ] [Detect/Mon] [Integration] [Docs]
```

3.8 Risk Identification, Assessment and Response

3.9 Quality Assurance and Quality Control

Quality Assurance (Process-level):

- Ty
pe
Scr
ipt
stri
ct
mo
de
en
abl
ed
for
all
so
urc
e fi
les
- ES
Lin
t wi
th r
ea
ct-
ho
ok
s a

nd
rea
ct-r
efr
es
h p
lug
ins

- Co
nsi
ste
nt
co
de
styl
e:
2-s
pa
ce
ind
ent
ati
on,
si
ngl
e q
uot
es,
se
mi
col
on
s
- I
mp
ort
dis
cipl
ine
: e
ver
y r
efe
ren
ce
d s
ym
bol
ha
s
a
ma
tch
ing
im
por
t
-

File

org
ani
zat
ion

by
co
he
sio
n (l
ib/,
co
mp
on
ent
s/,
pa
ge
s/,
ch
art
s/)

Quality Control (Product-level):

- npm
run
bu
ild
—
Pr
od
uct
ion
bu
ild
mu
st
co
mp
let
e
wit
ho
ut
err
ors
- npm
run
ty
pe
ch
ec
k
—
Ty
pe
Scr

ipt
co
mp
iler
m
ust
re
por
t z
ero
err
ors

- Visual
ual
ve
rifi
cati
ion
of
ea
ch
pa
ge
by
re-
rea
din
g s
our
ce
co
de
- Da
tab
as
e
mi
gra
tio
n v
erif
ied
vi
a
Su
pa
ba
se
M
CP
to
ols
- RLS
policies
ve
rifi

ed
via
se
cur
ity
po
stu
re
ch
ec
k

3.10 Agile/Scrum Development Approach

Justification for Agile over Waterfall:

The ML pipeline is inherently experimental — the effectiveness of different algorithms and feature combinations cannot be fully predicted upfront. Agile's iterative approach allows:

1.

Early
validation:

Ea
ch p
ipeli
ne s
tage
is i
nde
pen
dent
ly te
stab
le b
efor
e
the
next
sta
ge d
epe
nds
on it

2.

Fl
exib
ility:
Cha
rt d
esig
ns
and
pag
e la
yout
s ev
olve
d du
ring
dev
elop
men
t

3. Incremental delivery: Each page is a working increment, not a partial component

4. Risk reduction: Integration happens continuously, not at the end

Scrum Ceremonies Adapted:

- Sprint Planning: Decompose pipeline stages into stages

e-l
ev
el
del
ive
rab
les

- Dai
ly
St
an
du
p:
Tra
ck
pro
gre
ss
via
To
do
Wr
ite
too
l

- Sp
rint
R
evi
ew
: B
uil
d v
erif
ica
tio
n a
fter
ea
ch
pa
ge

- Sp
rint
R
etr
os
pe
ctiv
e:
Adj
ust
ap
pro
ac
h b

as
ed
on
bui
ld/t
yp
ec
he
ck
res
ult
s

3.11 Secure SDLC

The project integrates security into every development phase:

3.12 Change Control

Chapter Four — System Analysis, Design & Security

4.1 Existing System Analysis

Traditional authentication systems operate as follows:

1. User

sub
mits
cre
dent
ials
(use
rna
me/
pas
swo
rd, t
oke
n, bi
ome
tric)
2. S
yste
m v
erifi
es c
red
enti
als
agai
nst
stor
ed v
alue
s
3. A
cce
ss g

rant
ed
or d
enie
d
— b
inar
y de
cisio
n
4.

No
cont
inuo
us
mon
itori
ng
of p
ost-
auth
enti
cati
on b
eha
vior

5. C
omp
romi
sed
cred
enti
als
gran
t
full
acc
ess
until
ma
nual
ly re
vok
ed

Weaknesses:

-

No

be
ha
vio
ral
ver
ific
ati
on
aft
er i
niti

al
aut
he
nti
cat
ion

- Cannot detect attacker's using valid credentials
- Relies on periodic password changes or manual intervention
- No real-time alerting for suspicious

cio
us
act
ivit
y p
att
ern
s

4.2 Proposed System

AuthGuard ML introduces continuous behavioral monitoring:

1. Login
events

are
coll
ecte
d
and
stor
ed (
mim
ickin
g C
MU
data
set
stru
ctur
e)

2. Per-
user
beh
avio
ral b
asel
ines

are
esta
blis
hed
from
hist
oric
al
data

3. Each
new
logi
n ev
ent
is sc
ore
d ag
ains

t
the
user
's b
asel
ine

4. E
vent
s ex
cee
ding
a th
resh
old
are
flag
ged
as a
nom
alie
s wit
h se
verit
y

5. Alerts
are
gen
erat
ed
and
que
ued
for a
naly
st re
view

6. Real-

time
mo
nitor
ing
mod
e str
eam
s
and
scor
es e
vent
s
live

4.3 System Architecture

[illegible]

4.6.1 Entity Relationship

```
login_events (1) % % % % % !' (N) detected_anomalies
%
! "
alerts (1:1)

model_metrics (independent)
```

4.6.2 Schema

Table: login_events

Indexes: user_id, timestamp, is_anomaly

Table: detected_anomalies

Indexes: user_id, timestamp, severity, status

Table: alerts

Indexes: user_id, timestamp, acknowledged

Table: model_metrics

Indexes: model_name, trained_at

4.7 AI Architecture

[illegible]

4.8 Security Architecture

4.8.1 Row Level Security (RLS)

All four database tables have RLS enabled with the following policy pattern:

```
-- Example: login_events SELECT policy
CREATE POLICY "anon_select_login_events" ON login_events FOR SELECT
  TO anon, authenticated USING (true);
```

Each table has 4 separate policies (SELECT, INSERT, UPDATE, DELETE) scoped to TO anon, authenticated because the application is single-tenant without a sign-in screen. The anon-key frontend client requires anon role access to read and write data.

4.8.2 Threat Model

Threat 1: Credential Compromise (Primary Threat)

Threat 2: Database Direct Access

Threat 3: XSS via Chart Components

Threat 4: State Desynchronization

4.8.3 Security Controls

Chapter Five — AI/ML and Software Implementation

5.1 Dataset

Source: CMU Insider Threat Test Dataset (https://kithub.cmu.edu/articles/dataset/Insider_Threat_Test_Dataset/12841247/1)

Implementation: Due to the dataset's size and format (log files requiring server-side processing), the system generates synthetic login events that mirror the CMU dataset's structure and characteristics. The data generator (src/lib/dataGenerator.ts) creates:

- 15
us
ers
(U
SR
00
01
—
US
R0
01
5)
wit
h i
ndi
vid
ual
be
ha
vio
ral
pro
file
s
- 10

de
vic
es
(P
C-
00
1
-
PC
-01
0)

- 5 a
ctiv
ity
typ
es
(Lo
go
n,
Lo
gof
f,
HT
TP,
Fil
e,
Em
ail)

- Co
nfi
gur
abl
e ti
me
ra
ng
e (
7-6
0 d
ay
s,
def
aul
t
30)

- ~
3,0
00
ev
ent
s p
er
30-
da
y g
en
era

tion
n
run

- 5%

an
om
aly
inj
ect
ion
rat
e
wit
h
4 a
tta
ck
typ
es:
Off
-ho
urs
lo
gin
(m
idn
igh
t-3
am
or
10
pm
-mi
dni
ght
)
- Un
fa
mili
ar
de
vic
e a
cc
es
s
- Un
fa
mili
ar
IP
ad
dre
ss
-

We
ek
en
d l
ogi
n
for
we
ek
da
y-o
nly
us
ers

Per-User Profile Generation:
Each user is assigned:

- A t
ypi
cal
lo
gin
ho
ur (8-10a
m)
wit
h
Ga
us
sia
n n
ois
e (std
de
v 1
.5-2.5
)
- Ty
pic
al
wo
rki
ng
da
ys
(M
on-
Fri,
so
me
in
clu
de
Sa
tur

day)

- 1-3

family PCs

- One typical

IP address

- 2-7

session identifier day

5.2 Data Preprocessing

The data generator produces pre-processed events with:

- Temporal encoding: day_of_week (0-6), hour_of_day (0-23)
- Boolean flags:

is
_of
f_h
our
s (
bef
ore
7a
m
or
aft
er
7p
m),
is
_w
ee
ke
nd
(S
atu
rda
y/S
un
da
y)

- Ca
teg
ori
cal
fie
lds
: u
ser
_id
, p
c_i
d, i
p_
ad
dre
ss,
ac
tivit
y s
tor
ed
as
tex
t

- Gr
ou
nd
trut
h: i
s_
an
om

ally
bo
ole
an
lab
el
for
su
per
vis
ed
ev
alu
ati
on

- An
om
ally
sc
ore
: P
re-
co
mp
ute
d s
cor
e (
0.7
-1.
0
for
an
om
ali
es,
0
for
nor
ma
l)

Missing values: No missing values — all fields are populated during generation.

Data cleaning: Events are sorted chronologically after generation and anomaly injection.

5.3 Feature Engineering

The feature engineering engine (`src/lib/featureEngineering.ts`) transforms raw login events into 9-dimensional feature vectors:

Behavioral Profiling: For each user, the engine computes:

- Me
an
an
d s
tan
dar
d d
evi
ati

on
of l
ogi
n h
our
s

- Set
of t
ypi
cal
da
ys
- Fre
qu
en
cy
ma
ps
for
PC
s a
nd
IPs
- Total
event
co
unt

5.4 Model Development

Five unsupervised anomaly detection models are implemented in `src/lib/anomalyDetector.ts`:

5.4.1 Isolation Forest (Threshold Approximation)

```
predictions = features.map(f =>  
  f.hour_deviation > 2 || f.pc_familiarity < 0.1 ||  
  f.ip_familiarity < 0.1 || f.is_off_hours === 1  
);
```

Flags events with hour deviation > 2 & PC/IP familiarity, or off-hours login.

5.4.2 One-Class SVM (Threshold Approximation)

```
predictions = features.map(f =>  
  f.hour_deviation > 1.5 || f.pc_familiarity < 0.15 ||  
  f.day_deviation === 1  
);
```

More sensitive threshold on hour deviation; includes day deviation.

5.4.3 Local Outlier Factor (Threshold Approximation)

```
predictions = features.map(f =>  
  f.hour_deviation > 2.5 || f.pc_familiarity < 0.05 ||  
  f.ip_familiarity < 0.05  
);
```

Stricter thresholds — only flags extreme deviations.

5.4.4 Autoencoder (Threshold Approximation)

```
predictions = features.map(f =>
  f.hour_deviation > 1.8 || f.pc_familiarity < 0.08 ||
  f.ip_familiarity < 0.08 || f.is_off_hours === 1 ||
  f.day_deviation === 1
);
```

Combines multiple signals with moderate thresholds.

5.4.5 Robust Z-Score (Threshold Approximation)

```
predictions = features.map(f =>
  f.hour_deviation > 2 || f.pc_familiarity < 0.1 ||
  f.ip_familiarity < 0.1 || f.is_weekend === 1
);
```

Similar to Isolation Forest but includes weekend flag.

5.5 Model Training

The training process (`runTraining()` in `src/lib/store.ts`):

1. E
nsur
es f
eatu
res
are
com
pute
d (c
alls
com
pute
Feat
ures
) if
nee
ded
)
2. Calls
trainModels(features)
whi
ch r
uns
all
5 m
odel
app
roxi
mati
ons
3. Each
model
pro
duc
es p
redi

```

ctio
ns (
bool
ean
arra
y)
for
all f
eatu
re v
ecto
rs
4.      P
      redi
      ctio
      ns
      are
      eval
      uate
      d ag
      ains
      t gro
      und
      trut
      h la
      bels
5.      R
      esul
      ts st
      ore
      d in
      appl
      icati
      on s
      tate
      and
      pers
      iste
      d to
      mod
      el_
      metr
      ics t
      able

```

Training animation: The UI shows a progress bar with 8 steps (preparing, training each model, evaluating, computing metrics) with 250ms delays for visual feedback.

5.6 Anomaly Detection Engine

The detection engine (`detectAnomalies()` in `src/lib/anomalyDetector.ts`) scores each event against the user's behavioral profile:

Scoring Algorithm:

Total score is clamped to $[0, 1]$. Events with score ≥ 0.4 are classified as anomalies.

Severity Classification:

Reason Generation: Each detected anomaly includes human-readable reasons explaining which behavioral signals triggered the detection (e.g., "Unusual login time (2:00) — 3.5x normal").

5.7 Software Implementation

5.7.1 Technology Stack

5.7.2 File Structure

```
src/
% % % App.tsx           # Root component, page routing
% % % main.tsx          # Entry point
% % % index.css         # Tailwind directives
% % % lib/
%   % % % types.ts      # TypeScript interfaces
%   % % % supabase.ts   # Supabase client
%   % % % dataGenerator.ts # Synthetic data generation
%   % % % featureEngineering.ts # Feature extraction
%   % % % anomalyDetector.ts # Detection & model training
%   % % % store.ts       # State management
% % % components/
%   % % % Layout.tsx     # Sidebar + header layout
%   % % % charts/
%       % % % BarChart.tsx
%       % % % LineChart.tsx
%       % % % DonutChart.tsx
%       % % % Heatmap.tsx
%       % % % ScatterPlot.tsx
%       % % % GaugeChart.tsx
%       % % % RadarChart.tsx
%       % % % ConfusionMatrix.tsx
%       % % % MetricCard.tsx
%       % % % index.ts
% % % pages/
%   % % % DashboardPage.tsx
%   % % % DataCollectionPage.tsx
%   % % % EDAPage.tsx
%   % % % FeatureEngineeringPage.tsx
%   % % % ModelTrainingPage.tsx
%   % % % ModelEvaluationPage.tsx
%   % % % AnomalyDetectionPage.tsx
%   % % % MonitoringPage.tsx
```

5.7.3 State Management

The application uses a custom pub/sub state management pattern (src/lib/store.ts):

- Single source of truth: A module-level state object

holds

all
ap
plic
ati
on
sta
te

-

Pub/
sub
pa
ttern:
Component
ent
ss
ub
scr
ibe
via
a
sub
scr
ibe
(listener)
and
receive
updates
when
sets
State
()
is
called

-

No

external
depend
ency:

Avoid
s Read
ed
ux/
Zustand
overhead
for
asingle-
element
app

- Databases
c:loadDataFromDB()
loads persisted data
on dashboard
mount;
generateAndStoreData()
writes

to
DB
du
rin
g g
en
era
tio
n

5.7.4 Custom Chart Components

All 9 chart components are built from scratch using SVG — no chart library dependency:

5.8 Integration

Database Integration:

- Su
pa
ba
se
clie
nt i
niti
aliz
ed
in
src
/
lib/
su
pa
ba
se.
ts
usi
ng
en
vir
on
me
nt
var
iab
les
- Data
generation
wr
ite
s
to l
ogi
n_
ev
ent
s
in
bat
ch
es

of
50
0 r
ow
s

- Model metrics persisted to model_metrics table after training

- Dashboard loads existing data from database on mount via aloadDataFromDB()

Page Integration:

-

All
pa
ge
s s
ub
scr
ibe
to
the
ce
ntr
al
sto
re
an
d
re-
ren
der

on
sta
te
ch
an
ge
s

-

Na
vig
ati
on
but
ton
s b
et
we
en
pip
eli
ne
sta
ge
s g
uid
e t
he
us
er t
hro
ug
h t
he
wo
rkfl
ow

-

Alert
badge
in

sideband updates in real-time

as new waveforms are generated

5.9 Security Implementation

-

RLS Policies:

All 4 tables have RLS enabled with separate CRUD policies for admin, authenticated roles

-

For
eig
n
Ke
ys:
de
tec
ted
_a
no
ma
lies
.ev
ent
_id
!' lo
gin
_e
ve
nts
.id;
al
ert
s.a
no
ma
ly_i
d !'
det
ect
ed
_a
no
ma
lies
.id,
bo
th
wit
h
ON
D
EL
ET
E
CA
SC
AD
E

- No
Ha
rdc
od
ed
Se
cre
ts:
Su

pa
ba
se
UR
L a
nd
an
on
ke
y r
ea
d fr
om
im
por
t.m
eta
.en
v e
nvi
ron
me
nt
var
iab
les

•

Type
Safety
: Ty
pe
Scr
ipt
stri
ct
mo
de
pre
ve
nts
im
plic
it a
ny
typ
es;

all
fun
cti
on
par
am
ete
rs
ex
plic
itly
typ
ed

- No

XS
S
Ve
cto
rs:
Re
act
's
bui
lt-
in
es
ca
pin
g;
no
da
ng
ero
usl
yS
etl
nn
er
HT
ML
us
ag
e a
ny
wh
ere
in
the
co
de
ba
se

Chapter Six — Testing, Quality & Security

6.1 Unit Testing

Unit testing is performed through TypeScript's type system and build verification:

6.2 Integration Testing

6.3 System Testing

6.4 User Acceptance Testing (UAT)

6.5 AI Model Evaluation

Models are evaluated using the following metrics computed in `evaluateModel()`:

Confusion Matrix Components:

- True
Positives
(T
P):
A
no
ma
lou
s e
ve
nts
co
rre
ctly
fla
gg
ed
- Fal
se
Po
siti
ve
s (F
P):
No
rm
al
ev
ent
s i
nc
orr
ect
ly fl
ag
ge
d
- True
Negatives
(T
N):
N
or
ma
l e
ve
nts
co
rre
ctly
pa
ss
ed
- Fal

se
Ne
gat
ive
s (
FN
):
An
om
alo
us
ev
ent
s
mi
ss
ed
Metrics:

- $$\text{Accuracy} = \frac{TP + TN}{TP + FP + TN + FN}$$
- $$\text{Precision} = \frac{TP}{TP + FP}$$
- $$\text{Recall} = \frac{TP}{TP + FN}$$
- F1-

$$\frac{Score = 2 \times (Precision \times Recall)}{(Precision + Recall)}$$

- RO C-AUC = 0.5 + (Recall - FPR) / 2

Feature Importance (estimated):
Model Comparison Summary:

6.6 Security Testing

6.7 Vulnerability Assessment

Chapter Seven — Deployment, Monitoring & Project Control

7.1 Deployment

Platform: Bolt.new — a cloud-based development and deployment platform for Vite + React applications.

Deployment Configuration:

- Build command

mand:
np
m r
un
bui
ld (
Vit
e p
rod
uct
ion
bu
ild)

- Ou
tpu
t di
rec
tor
y:
dis
t/

- En
vir
on
me
nt:
Pr
e-c
onf
igu
red
S
up
ab
as
e c
red
ent
ials
in
.en
v

- Ho
sti
ng:
St
ati
c fil
e h
ost
ing
wit
h _
red
ire

cts

for

SP

A r

out

ing

Database Deployment:

-

Sc

he

ma

ap

pli

ed

via

m

cp

—

su

pa

ba

se

—

ap

ply

_m

igr

ati

on

M

CP

to

ol

-

Mi

gra

tio

n fi

le:

20

26

09

15

21

53

07

_cr

eat

e_

an

om

aly

_d

ete

cti

on

_s

ch

em

a.s
ql

- 4 t
abl
es,

16
RL
S
pol
icie
s,
11
ind
ex
es
cre
ate
d
- Mi
gra
tio
n
is i
de
mp
ote
nt (
us
es
IF
NO
T
EX
IS
TS
an
d
DR
OP
P
OL
IC
Y
IF
EX
IS
TS
)

7.2 Configuration

7.3 Monitoring

The Monitoring page (src/pages/MonitoringPage.tsx) provides:
Alert Dashboard:

- Total
alert

s c
ou
nt

- Act
ive
(u
na
ck
no
wle
dg
ed)
al
ert
s c
ou
nt
- Ac
kn
owl
ed
ge
d a
lert
s c
ou
nt
- Cri
tic
al
se
ver
ity
ale
rts
co
unt

Visualizations:

- Do
nut
ch
art:
Al
ert
s
by
se
ver
ity
(lo
w,
me
diu
m,
hig

h,
crit
ical
)

- Line
chart:
Al
ert
tim
eli
ne
(al
ert
s p
er
da
y)

- Bar

ch
art:
Al
ert
s p
er
us
er (
top

10)

Alert Queue:

- Filt
era
ble

by
sta
tus
(Al
l,
Op
en,
Ac
kn
owl
ed
ge
d)

- Ex
pa
nd
abl
e r
ow
s w

ith
ale
rt d
eta
ils

- Col
or-
co
de
d
by
se
ver
ity
(re
d/o
ran
ge/
am
ber
/gr
ay
bor
der
)
- Ac
kn
owl
ed
ge
but
ton

for
ea
ch
ale
rt

7.4 Security Monitoring

Real-Time Detection:

The Anomaly Detection page supports a live monitoring mode that:

- Ge
ner
ate
s n
ew
lo
gin
ev
ent
s e
ver
y 1
.5
se

cond
s

- Score
se
ach
event
against
user
behavioral
profiles

- Displays
a live
event
stream
with
anoma
ly scores

- Col
or-
code
se
vents

by
an
om
aly
st
atus
(red/
gre

en)

-

Shows every bad guess for flagged anomalies

Alert Generation:

When anomalies are detected:

1.

Each anomaly is stored as a Detected Anomaly record with features snapshot

2.

An Alert is generated with a human-readable message

3.

Sever

ity
is a
ssig
ned
bas
ed
on a
nom
aly
scor
e (cr
itical
/hig
h/m
ediu
m/
low)

4.

Alerts
appear
in
the
side
bar
bad
ge
and
mon
itori
ng p
age

5.

A
naly
sts
can
ack
now
ledg
e al
erts
to tr
ack
revi
ew
stat
us

7.5 Project Progress and Control

Progress Tracking:

-

To
do
Wr
ite
too
l u
se
d
to t

rac
k
13
tas
ks
acr
os
s t
he
de
vel
op
me
nt l
ife
cyc
le

- Ea
ch
tas
k
ma
rke
d
as
co
mp
let
ed
im
me
dia
tel
y a
fter
fin
ishi
ng

- Bui
ld
an
d t
yp
ec
he
ck
run
aft
er
all
co
de
ch
an
ge
s

Quality Gates:

7.6 Incident Response and Recovery

Incident Response Procedure (for security-related projects):

Recovery Considerations:

- Data
tab
as
e b
ac
ku
ps:
S
up
ab
as
e p
rov
ide
s a
uto
ma
ted
da
ily
ba
ck
up
s
- Data
integrity
: F
ore
ign
ke
y c
on
str
ain
ts
wit
h
CA
SC
AD
E
pre
ve
nt
orp
ha
ne
d r
ec
ord
s
- St

ate
re
co
ver
y: l
oa
dD
ata
Fro
m
DB
() r
est
ore
s a
ppl
ica
tio
n s
tat
e fr
om
da
tab
as
e
on
pa
ge
rel
oa
d

•

Mo
del
ret
rai
nin
g:
Mo
del
s c
an
be
retr
ain
ed
at
an
y ti
me
vi
a t
he
Mo
del
Tr
ain
ing
pa

Chapter Eight — Conclusion

8.1 Summary

AuthGuard ML is a complete AI-based user authentication anomaly detection system that demonstrates the full machine learning project lifecycle — from data collection through deployment and monitoring. The system generates synthetic login events modeled on the CMU Insider Threat Test Dataset, engineers 9 behavioral features per event, trains 5 anomaly detection algorithms, evaluates them using standard ML metrics, and provides both batch and real-time anomaly detection with an alert management workflow.

The web application consists of 8 interactive pages with custom-built SVG chart components (no chart library dependency), a responsive sidebar layout, and persistent storage via Supabase (PostgreSQL) with Row Level Security on all tables. The entire system runs in the browser with a managed database backend, requiring no server-side infrastructure.

8.2 Objectives Achieved

8.3 Conclusion

The project successfully demonstrates that machine learning can be applied to authentication anomaly detection by establishing per-user behavioral baselines and flagging deviations. The system addresses the critical gap between point-in-time authentication and continuous behavioral monitoring, providing security analysts with explainable anomaly detection — each flagged event includes human-readable reasons explaining which behavioral signals triggered the detection.

The implementation of 5 different anomaly detection algorithms with comparative evaluation provides insight into the trade-offs between sensitivity (recall) and precision across different approaches. The custom-built visualization suite enables intuitive exploration of login patterns, model performance, and detected anomalies without relying on external charting libraries.

8.4 Limitations

1. S
ynth
etic
Dat
a: T
he s
yste
m u
ses
proc
edu
rally
gen
erat
ed d
ata
rath
er t
han
real-
worl
d lo
gs,
whic
h m
ay
not

capture
all behavioral
complexities
of actual
insider
threats.

2.

Model
Approximations

: There are
many detection
algorithms
used
threshold-based
approximations
rather than
full library
implementations (e.g.,
scikit-learn's Isolation
Forests)
to enable
in-browser

re-
execution.

3. Single-
Tenant:
No user
authentication;
suitable
for
demonstration
but
not
multi-
organiza-
tion
deployment.

4. Feature
Scope: Limited
to
temporal
and
device-
based
features;
does
not
include
session
duration,
data
access
patterns

erns
, or
naviga-
tion habi-
tats.

5. Scalability: In-browser computation suitable for ~5000 events; production scale would require server-side inference.

6. No Real-Time Database Subscription: Live monitoring uses client-side event generation

on r
athe
r th
an
Sup
aba
se
real-
time
sub
scri
ptio
ns.

8.5 Recommendations

1. In
tegr
ate
Rea
l C
MU
Dat
a: P
roce
ss
the
actu
al C
MU
Insi
der
Thr
eat
data
set
log f
iles
thro
ugh
a se
rver
-sid
e pi
peli
ne
for
mor
e re
alisti
c be
havi
oral
patt
erns
.
2. Add

Use

Authentication:
Implement Supabase
email/password
authentication
to support
multi-tenant
deployment
with

per-user
data isolation.

3. Server-Side
ML: Deploy
anomaly
detection
algorithm
as
Supabase
Edge
Function
using
Python
ML

libraries
for production-
grade inference.

4. Expand Feature Set: Added session duration, data access volume, navigation patterns, and typing dynamics for richer behavioral profiling.

5. Real-time Session Descriptions: Use Session Database real-time

channel
s to
sub
scri
be
to n
ew l
ogin
eve
nts f
rom
exte
rnal
syst
ems

6. A
uto
mat
ed
Retr
aini
ng: I
mpl
eme
nt s
che
dule
d m
odel
retr
aini
ng
as n
ew
beh
avio
ral d
ata
acc
umu
late
s, wi
th
drift
dete
ctio
n.

7. In
tegr
atio
n A
PIs:
Pro
vide
RE
ST
API
end

points
for integration
detecting
results
with external
SIE
Ms
systems.

8. Mobile Alerts:
Ad push
notification
or email
alerts
delivery
for critical-
severity
anomalies.

8.6 Future Work

- Deep Learning Models:
Implement LSTM

-based sequential anomaly detection to capture temporal portal sequence of user behavior.

- Federated Learning: Enable cross-organizational model training without sharing

ng
ra
w
be
ha
vio
ral
dat
a.

-

Ex
pla
ina
ble

AI:
Int
egr
ate
S
HA
P
or
LI
ME

for
de
ep
er
mo
del
ex
pla
ina
bilit
y b
ey
on
d t
he
cur
ren
t re
as
on
ge
ner
ati
on.

-

Ad
ver
sar
ial
Ro
bu
stn
es
s:

Test detection against adversarial behavior designed to mimic legitimate user patterns.

- Multimodal Detection: Combine behavioral or network traffic analysis

d e
nd
poi
nt t
ele
me
try
for
mu
lti-l
ay
ere
d d
ete
cti
on.

- Aut
om
ate
d
Re
sp
on
se:
Im
ple
me
nt
aut
om
ate
d a
cc
ou
nt
su
sp
en
sio
n
or
ste
p-
up
aut
he
nti
cat
ion

for
crit
ical
-se
ver
ity
an
om
ali

es.

8.7 Lessons Learned

1. Feature engineering is critical: The quality of behavioral features (hour deviation, PC/IP familiarity) had more impact on detection quality than the choice of algorithm.

2. Explainability matters: Human-readable anomaly reports

also
ns
are
ess
enti
al
for a
naly
st tr
ust
and
effic
ient
inve
stig
atio
n

—
a sc
ore
alon
e is
insu
ffici
ent.

3. C
usto
m C
hart
s
vs L
ibrar
ies:
Buil
ding
SV
G c
hart
s fro
m s
crat
ch p
rovi
ded
full
cont
rol o
ver
styli
ng
and
red
uce
d bu
ndle
siz
e,
but r
equi
red

more development effort.

4.

State Management

Simplicity:

A custom pub/sub pattern was sufficient

for a single-tenant app;

Redux would have been unnecessary overhead.

5.

Database-First Design: Designing the schema with RLS policies upfront

pre
vent
ed a
cce
ss c
ontr
ol is
sue
s
that
wou
ld
be d
iffic
ult
to r
etro
fit.

6. It
erati
ve
Dev
elop
men
t: Th
e A
gile
app
roac
h of
buil
ding

one
pipe
line
stag
e at
a ti
me,
with
buil
d ve
rific
atio
n aft
er e
ach,
cau
ght i
ssu
es e
arly.

7. Type

Saf
ety
as T
esti

ng:
Typ
eSc
ript
stric
t mo
de s
erve
d
as
a for
m
of
unit
testi
ng,
catc
hing
inte
rfac
e mi
sma
tche
s at
com
pile
time
.

Appendix A: Technology Stack Summary

Appendix B: Database Migration Summary

Migration: 20260915215307_create_anomaly_detection_schema.sql

Appendix C: Feature Vector Specification

Appendix D: Anomaly Scoring Algorithm

Classification thresholds: Low (<0.4), Medium ("e0.4), High ("e0.6), Critical ("e0.8)
